

Práctica 2:

Expresiones regulares

Grado en Ingeniería Informática
Teoría de la Computación

Introducción

Una expresión regular es una secuencia de caracteres que define un patrón de búsqueda sobre cadenas de símbolos. Para un alfabeto de símbolos Σ , las siguientes constantes son expresiones regulares:

$\emptyset$	el conjunto vacío (no encaja con ninguna cadena)
ε	la cadena vacía
a	$\forall a \in \Sigma$

Y dadas dos expresiones regulares r y s , las siguientes operaciones sobre ellas definen nuevas expresiones regulares:

$r \cdot s$	concatenación
r^*	repetición 0 o más veces
$r + s$	or lógico o alternativa
$r \& s$	and lógico

Por ejemplo, a^*b encaja con las cadenas b , ab , aab , etc. Para simplificar las expresiones regulares con las que trabajaremos, vamos a considerar tres casos adicionales:

$\cdot$	cualquier símbolo
$\hat{c}$	cualquier símbolo distinto de c
$a - b$	cualquier símbolo entre a y b

que serán expresables en términos de los casos anteriores como el *or* de todos los símbolos de Σ (en el caso de $\cdot$), el *or* de todos los símbolos de Σ excepto c (en el caso de $\hat{c}$), y el *or* de todos los símbolos en un rango (en el caso de $a - b$).

El estándar Posix especifica una sintaxis para expresiones regulares que utilizan muchas aplicaciones Unix. En esta práctica vamos a trabajar con un subconjunto de esta sintaxis:

a	carácter a
$\cdot$	cualquier carácter
$[\]$	cualquiera de los caracteres entre los corchetes (ejemplo: $[abc]$ encaja con a , b o c) (ejemplo: $[a - z0 - 9]$ encaja con cualquier número o carácter en minúscula)
$[\hat{ }]$	cualquier carácter excepto los especificados (ejemplo: $[^ab]$ encaja con cualquier carácter excepto a y b)
$*$	0 o más repeticiones
$(\)$	agrupa subexpresiones (ejemplo: $(ab)^*$ encaja con ε , ab , $abab$, etc.)
$+$	1 o más repeticiones (ejemplo: a^+ equivale a aa^*)
$?$	0 o 1 veces
$ $	alternativa (ejemplo: $abc def$ encaja con abc o con def)

Por ejemplo, el comando `grep` escribe las líneas de un fichero que encajan con una expresión regular:

```
grep -E "let" src.ml
líneas con definiciones en un fichero .ml

grep -E "let ([^r]|r[^e]|re[^c])" src.ml
líneas con definiciones no recursivas

grep -E "let.*in" src.ml
líneas con un let-in
```

Derivada de una expresión regular

Para determinar si una cadena de texto encaja con una expresión regular, vamos a utilizar las derivadas de las expresiones regulares.

La derivada de un conjunto de cadenas S con respecto a un símbolo a es el conjunto de cadenas de S que empiezan por a , pero sin el símbolo inicial a . Para el conjunto de cadenas aceptadas por una expresión regular, el conjunto de cadenas derivadas respecto a un símbolo es expresable también como una expresión regular.

Para calcular la derivada de una expresión regular r respecto a un carácter c , definimos dos funciones. La función $\nu(r)$, cuyo valor es:

$$\nu(r) = \begin{cases} \varepsilon & \text{si } r \text{ acepta la cadena vacía} \\ \emptyset & \text{si no la acepta} \end{cases}$$

y se define como:

$$\begin{aligned} \nu(\emptyset) &= \emptyset \\ \nu(\varepsilon) &= \varepsilon \\ \nu(a) &= \emptyset \\ \nu(\cdot) &= \emptyset \\ \nu(\hat{a}) &= \emptyset \\ \nu(a - b) &= \emptyset \\ \nu(\hat{a} - b) &= \emptyset \\ \nu(r \cdot s) &= \begin{cases} \varepsilon & \text{si } \nu(r) = \varepsilon \text{ y } \nu(s) = \varepsilon \\ \emptyset & \text{en otro caso} \end{cases} \\ \nu(r^*) &= \varepsilon \\ \nu(r + s) &= \begin{cases} \varepsilon & \text{si } \nu(r) = \varepsilon \text{ o } \nu(s) = \varepsilon \\ \emptyset & \text{en otro caso} \end{cases} \\ \nu(r \& s) &= \begin{cases} \varepsilon & \text{si } \nu(r) = \varepsilon \text{ y } \nu(s) = \varepsilon \\ \emptyset & \text{en otro caso} \end{cases} \end{aligned}$$

y la función que calcula la derivada de una expresión regular r con respecto a un carácter a , $\partial_a(r)$, que se define como:

$$\begin{aligned}
\partial_a(\emptyset) &= \emptyset \\
\partial_a(\varepsilon) &= \emptyset \\
\partial_a(a) &= \varepsilon \\
\partial_a(c) &= \emptyset \quad \text{si } c \neq a \\
\partial_a(\hat{a}) &= \emptyset \\
\partial_a(\hat{c}) &= \varepsilon \quad \text{si } c \neq a \\
\partial_a(c - d) &= \varepsilon \quad \text{si } c \leq a \leq d \\
\partial_a(c - d) &= \emptyset \quad \text{si } a < c \text{ o } a > d \\
\partial_a(\hat{c} - d) &= \varepsilon \quad \text{si } a < c \text{ o } a > d \\
\partial_a(\hat{c} - d) &= \emptyset \quad \text{si } c \leq a \leq d \\
\partial_a(.) &= \varepsilon \\
\partial_a(r \cdot s) &= \partial_a(r) \cdot s + \nu(r) \cdot \partial_a(s) \\
\partial_a(r^*) &= \partial_a(r) \cdot r^* \\
\partial_a(r + s) &= \partial_a(r) + \partial_a(s) \\
\partial_a(r \& s) &= \partial_a(r) \& \partial_a(s)
\end{aligned}$$

Una cadena formada por los caracteres $c_1c_2\dots c_n$ encaja con una expresión regular r si la derivada de r para toda la cadena acepta ε , esto es, si se cumple que:

$$\nu(\partial_{c_n}(\dots \partial_{c_2}(\partial_{c_1}(r)))) = \varepsilon$$

Enunciado

Con la práctica se incluye un *parser* que interpreta expresiones regulares en formato Posix, y construye valores de tipo `Regexp.regexp` utilizando las funciones especificadas en `regexp.mli`:

```

type symbol = ...
type regexp = ...

val symbol_of_char  : char -> symbol      (* un símbolo es un carácter *)
val symbol_of_range : char -> char -> symbol (* o cualquiera en un rango *)

val empty          : regexp              (* conjunto vacío *)
val empty_string   : regexp              (* cadena Vacía *)
val single         : symbol -> regexp     (* un único carácter o rango *)
val except         : symbol -> regexp     (* todo carácter excepto ... *)
val any            : regexp              (* cualquier carácter *)
val concat         : regexp -> regexp -> regexp (* concatenación *)
val repeat         : regexp -> regexp     (* repetición *)
val alt            : regexp -> regexp -> regexp (* or *)
val all            : regexp -> regexp -> regexp (* and *)

```

Se pide lo siguiente:

1. Defina en `regexp.ml` el tipo `symbol` para representar caracteres individuales o rangos de caracteres, y el tipo `regexp` que representa expresiones regulares. Defina las funciones especificadas en `regexp.mli` que construyen valores de esos tipos. Modifique el fichero `regexp.mli` para incluir las definiciones de los tipos `symbol` y `regexp`.

2. Defina en un nuevo fichero `derive.ml` las funciones especificadas en `derive.mli`:

- `regexp_of_string : string -> Regexp.regexp`, que construye un valor de tipo `regexp` a partir de un `string` con una expresión regular en formato Posix utilizando el parser:

```
let regexp_of_string s =  
  Regexp_parser.main Regexp_lexer.token (Lexing.from_string s)
```

- `nullable : Regexp.regexp -> Regexp.regexp`, que para una expresión regular r devuelva $\nu(r)$.
- `derive : char -> Regexp.regexp -> Regexp.regexp`, que para un carácter c y una expresión regular r devuelva $\partial_c(r)$.
- `matches_regexp : string -> Regexp.regexp -> bool`, que calcula si una cadena encaja con una expresión regular.
- `matches : string -> string -> bool`, donde `matches str1 str2` calcula si `str1` encaja con la expresión regular en formato Posix `str2`.

3. Comando `grep`. El material proporcionado incluye también un programa `grep` que, utilizando los módulos de la práctica, implementa una versión simplificada del comando `grep -E`. Una vez que tenga implementados `regexp.ml` y `derive.ml`, puede compilarlo con `make grep` y probarlo con `./grep`.

4. Optimización (apartado opcional). Varias de las reglas de derivación producen una expresión regular más compleja que la original, y el rendimiento con cadenas largas se resiente. Añada a `derive.ml` una función

```
simplify : Regexp.regexp -> Regexp.regexp
```

que simplifique recursivamente una expresión regular, teniendo en cuenta las siguientes reglas:

- $\emptyset \cdot r = \emptyset, r \cdot \emptyset = \emptyset$
- $r \cdot \epsilon = r, \epsilon \cdot r = r$
- $\epsilon^* = \epsilon$
- $\emptyset^* = \emptyset$
- $\emptyset + r = r, r + \emptyset = r, r + r = r$
- $\emptyset \& r = \emptyset, r \& \emptyset = \emptyset$

y modifique las funciones `matches` y `matches_regexp` para que simplifiquen la expresión regular después de cada derivación. Compile de nuevo el programa `grep` y compruebe que sigue funcionando correctamente.